Assignment 7 — Solution Key

Microprogrammed Control Unit Design

Auqib Hamid Lone

Department of Computer Science & Engineering, GCET Ganderbal

August 8, 2026

7.1 Conceptual

Q1. The mapping unit converts the opcode currently in `IR` into the microaddress where *that instruction's* microprogram begins in `CM`, and loads it into `MPC` on every fetch. Without it, `MPC` would always start at address 0 regardless of which instruction is actually in `IR` — so **every instruction would execute whichever microprogram happens to occupy `CM` row 0**, regardless of its actual opcode. This is the microprogrammed analogue of a missing opcode decoder: a shared `CM` can only serve every instruction correctly if something translates opcode into starting address.

Q2. Width: horizontal encoding needs one bit per control signal, so the control word is as wide as the total number of signals in the machine (100–400 bits is typical at real-ISA scale); vertical encoding groups mutually-exclusive signals into fields, needing only $\lceil \log_2 n \rceil$ bits per n-way group, so the total word is much narrower. **Delay:** horizontal signals are valid the instant `CM` is read, since each bit *is* the signal directly, with no logic in between; vertical signals require an extra decoder stage between `CM`'s output and the datapath, so they become valid one decode delay *after* `CM` is read.

7.2 Numerical

Q3. Vertical: $\lceil \log_2 20 \rceil =$ **5 bits** (5 bits address up to 32 values, the smallest power of two ≥ 20; 4 bits would only reach 16, insufficient). **Horizontal: 20 bits**, one per signal.

Q4. $3 + 3 + 2 =$ **8 total `CM` rows** (assuming, as stated, that no rows are shared between the three instructions' microprograms).

7.3 Design

Q5. `SUB` has the identical shape to the lecture's `ADD` microprogram, with the ALU field changed to subtract at row 1:

Microaddress	RTL	Control word (signals asserted)	Next
0	$Y \leftarrow$ [R5]	$R5_{\text{out}}, Y_{\text{in}}$	1
1	$Z \leftarrow [Y] - [\text{R6}]$	$R6_{\text{out}}$, ALU=subtract, Z_{in}	2
2	$R4 \leftarrow [Z]$	$Z_{\text{out}}, R4_{\text{in}}$, END	fetch

As in the lecture's `ADD` example, only *where* the three control steps live has changed (three rows of `CM` instead of three rows of a hardwired state table); the RTL and signal content are otherwise

identical in structure to the `ADD` case, differing only in the register names and the ALU function selected at microaddress 1.

Q6. One valid grouping (following the lecture's worked example, which used Bus-source / Destination / ALU-operation fields):

- **Bus-source field** — selects which register drives the bus this step: $\{R2_{\text{out}}, R3_{\text{out}}, Z_{\text{out}}\}$, 3 mutually exclusive options (plus a "none" encoding) $\Rightarrow \lceil \log_2 4 \rceil = 2$ bits.
- **Destination field** — selects which register loads from the bus/ALU: $\{Y_{\text{in}}, Z_{\text{in}}, R1_{\text{in}}\}$, 3 options (plus "none") $\Rightarrow$ 2 bits.
- **ALU-operation field** — here only one operation (add) is used in this microprogram, so 1 bit suffices to indicate "add active / inactive."
- **End field** — a single dedicated bit, since it is not mutually exclusive with the others (it can co-occur with a destination write on the final step).

Total: $2+2+1+1 = 6$ **bits**, versus **8 bits** for horizontal encoding of the same 8 signals — a modest saving on this toy example, which the lecture notes compounds significantly at real instruction-set scale, where the horizontal count grows into the hundreds of bits while the vertical count grows only logarithmically with each group's size.